

10.3.4 YOLO 目标检测与 ROS 2 节点接入

课程信息

- 环境：Windows 10、WSL2 Ubuntu 24.04、ROS 2 Jazzy；可选 Gazebo Harmonic 相机；**不强制安装** PyTorch / ultralytics。
- **认知主线**（先于模型名词）：图像从哪来 → 怎么从图里找到目标 → 模型怎么装进这台电脑 → 结果怎么交给 ROS 2。
- 动手目标：合成源零依赖跑通 → 真实 YOLO 视频推理 → ONNX 导出与推理 → 可选 Gazebo 相机接入；**CP4** (Checkpoint 4) 验收。
- 主路径：perception.launch.py + /mecamind/detections；兜底：HSV 颜色检测，没装 ML 依赖也能完成全部 LAB 与 CP4。

路径约定：本讲义假设项目位于 `~/mecamind_ros2`。若路径不同，请自行替换。

前置：第三节导航已能跑通（`scripts/test_cp3.sh`），会用 `ros2 topic echo` / `ros2 launch`。

本节不要求跟随控制或多线程细节；第五节会复用本节的检测消息接口，在其上做「跟着人走」。

先读这段：本节主线（比背 YOLO 更重要）

前三课机器人已经有了「腿」：会动、会建图、会导航到命名房间。

第四节要给它装「眼睛」——从画面里认出目标，并把结果用 ROS 2 话题交给下游。

很多人一上来就搜「怎么装 YOLO」，结果卡在 pip、CUDA、模型下载上，整节课都没看见一帧带框画面。

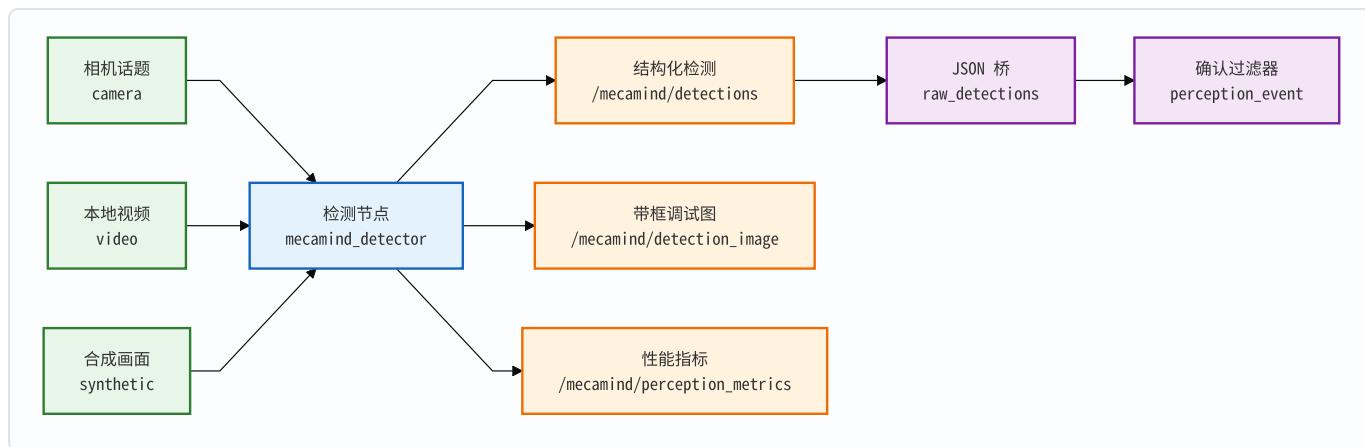
本课刻意反过来：先保证「眼睛能睁开」（哪怕只用颜色阈值），再逐步换成真正的神经网络；缺依赖就自动降级，课堂永不熄灯。

感知本身只回答四个问题，按时间顺序就是：

- ① 图像从哪来？ → camera / video / synthetic 三种输入源
- ② 怎么从图里找到目标？ → HSV 颜色阈值 vs YOLO 神经网络，两条路线
- ③ 模型怎么装进这台电脑？ → .pt 到 .onnx 导出，为什么部署要「翻译」
- ④ 结果怎么交给 ROS 2？ → 自定义消息、归一化坐标、调试图、JSON 兼容桥

读下文时请按这个顺序；`ultralytics`、`onnxruntime`、`letterbox`、`NMS` 只是实现这四问的工具。

把整节课压缩成一张图——**三进三出，中间一个检测器**：



- **绿色**：图像从哪进系统（主线①）
- **蓝色**：检测节点本体（主线②③）
- **橙色**：三种标准输出（主线④）
- **紫色**：兼容旧契约 + 连续帧确认（下游，第五节会接着用）

三级兜底可以记成一句口诀：

市电 → 柴油发电机 → 手电筒：ultralytics 有就用；没有就 onnx；再没有就 HSV。课堂永不熄灯。

1. 本节学习目标

读完并做完 LAB 后，应当能够：

1. 用自己的话讲清「眼睛」四问：图像从哪来、怎么找目标、模型怎么部署、结果怎么进 ROS 2。
2. 解释 `.pt`、`.onnx`、HSV 分别是什么、各自依赖什么、课堂为何要三级兜底。
3. 说清 `Detection2D` 为何用归一化坐标（0~1），以及这对第五节跟随控制意味着什么。
4. 用 `perception.launch.py` 分别跑通 `synthetic+hsv`、`video+ultralytics`、`video+onnx` 三条路径。
5. 用 `scripts/test_cp4.sh` 完成 CP4，并能从输出里读出当前用的是哪个后端。

（新名词全称、本课新增文件清单，集中放在文末 §2.7 速查，需要时再翻。）

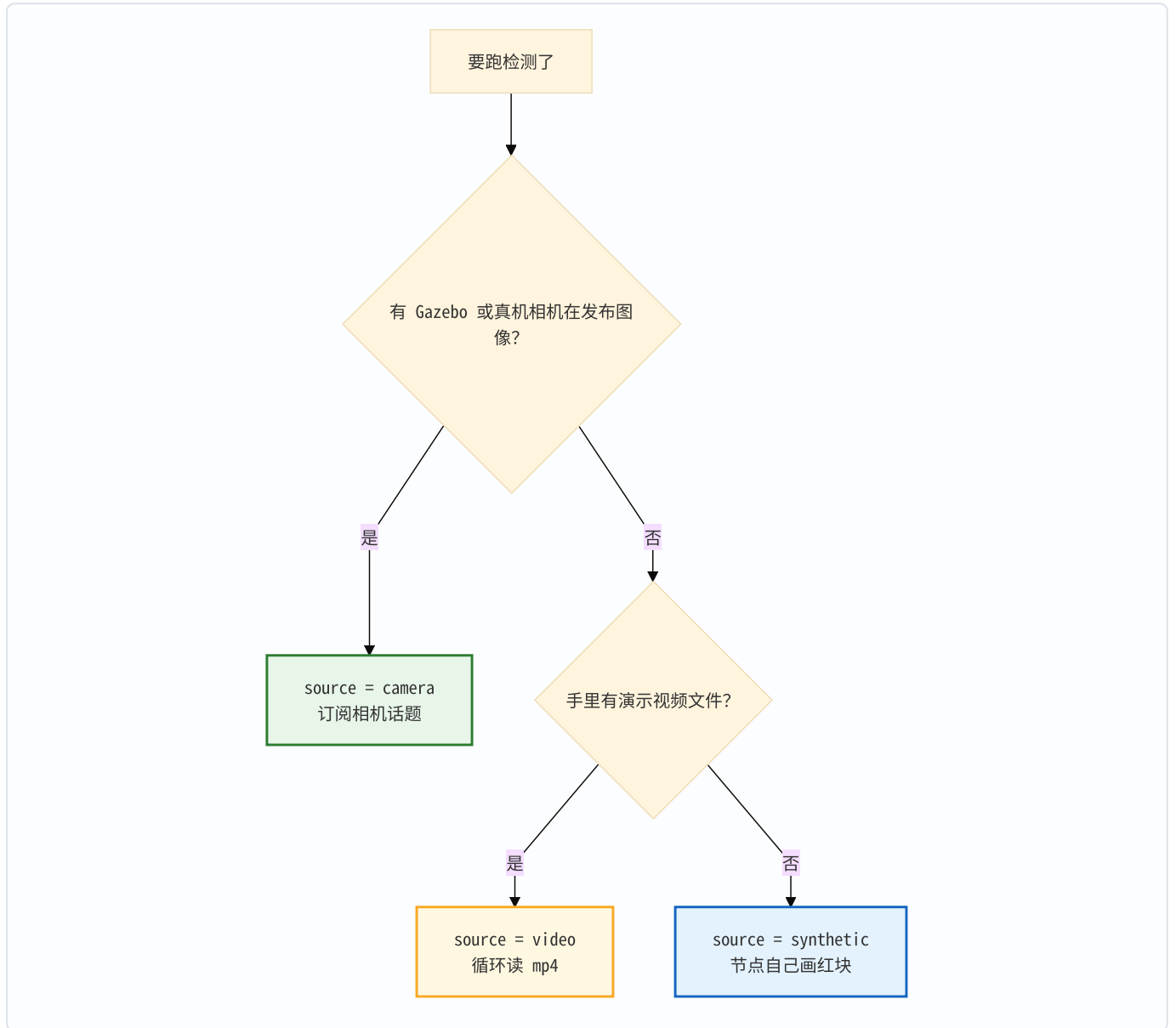
2. 主线详解

2.1 主线①：图像从哪来？

检测器再聪明，没有图像也是瞎子。本课检测节点用参数 `source` 在三种输入之间切换：

source	图像从哪来	典型用途	依赖
synthetic	节点自己画：灰底 + 左右晃的红色人形剪影	零依赖冒烟、讲 HSV、无摄像头也能上课	无
video	本地 mp4 文件循环播放	真实 YOLO 演示、可复现的课堂实验	视频文件
camera	订阅 /camera/image_raw (或你指定的话题)	Gazebo / 真机相机，机器人真正「睁眼」	相机发布者

选择逻辑可以记成一张判断图：



为什么必须有 synthetic?

课堂环境千差万别：有人没开 Gazebo、有人笔记本没摄像头、有人公司网络下不了演示视频。若没有合成源，这些同学第一节 LAB 就会卡死。合成源的契约很简单：

- 固定分辨率 640×480 ；
- 灰底 + 一条「地面线」；
- 一个红色矩形 + 红圆「头」，水平方向按正弦左右晃。

它故意做成「一眼就能看出的大红块」——正好喂给后面的 HSV 兜底后端。
YOLO 在 COCO 上训练，认的是真人、车、猫，**认不出这块红方块**——这一点后面 § 2.5 会当反面教材讲。

相关参数（启动时用 `source:=...` 覆盖）：

参数	默认	含义
<code>source</code>	<code>synthetic</code>	输入源
<code>camera_topic</code>	<code>/camera/image_raw</code>	camera 模式下订阅的话题
<code>video_path</code>	空	video 模式下的 mp4 路径
<code>frame_id</code>	<code>camera_optical_frame</code>	发布检测时写入 header 的坐标系名

类比：三种 source 像三种「眼睛硬件」——真机摄像头、事先录好的监控录像、以及课堂上用记号笔画出来的示意板。检测算法不应关心画面从哪来，只关心「拿到一帧 BGR 图」。

2.2 主线②：怎么从图里找到目标？——先从最朴素的 HSV 说起

在碰神经网络之前，先回答一个更朴素的问题：**如果目标永远是「红色的」，要不要训练模型？**

多数情况下，**不要**。用颜色阈值就能搞定，而且代码不超过几十行、零额外 pip 依赖。本课的 HSV 后端就是这个角色——手电筒级别的照明，但足够照亮整节课。

2.2.1 三个字母：H、S、V 到底是什么

摄像头给你的图通常是 **BGR**（蓝、绿、红三个通道叠在一起）。
对「找红色」这件事，BGR 很别扭：同一块红，亮一点暗一点，三个通道的数值都会变，很难用一组固定阈值框住。

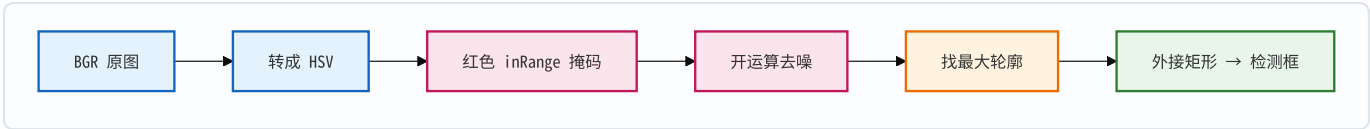
换成 **HSV** 就直观多了：

通道	英文	中文直觉	取值习惯（OpenCV）
H	Hue	色相：红、橙、黄、绿……在色环上的位置	0~180（OpenCV 把 360° 折半）
S	Saturation	饱和度：颜色有多「浓」，0 接近灰白	0~255
V	Value	明度：有多亮	0~255

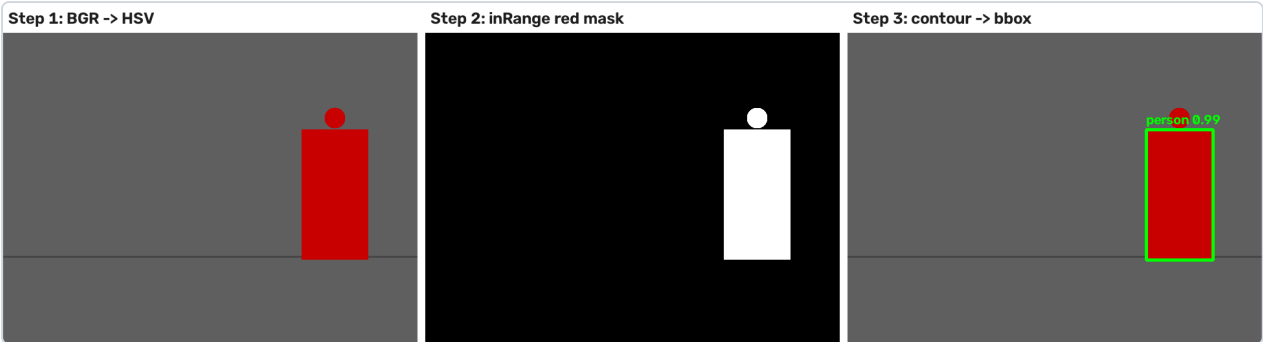
找红色时，你主要卡 **H**（色相落在红色区间），再要求 **S、V** 别太低（排除灰墙、阴影）。
注意：红色在色环上骑在 0 度两侧，所以 OpenCV 里通常要**两段**区间并起来：接近 0 的一段 + 接近 180 的一段。

2.2.2 四步流水线：从像素到检测框

本课 `HsvBackend.detect` 的真实流程：

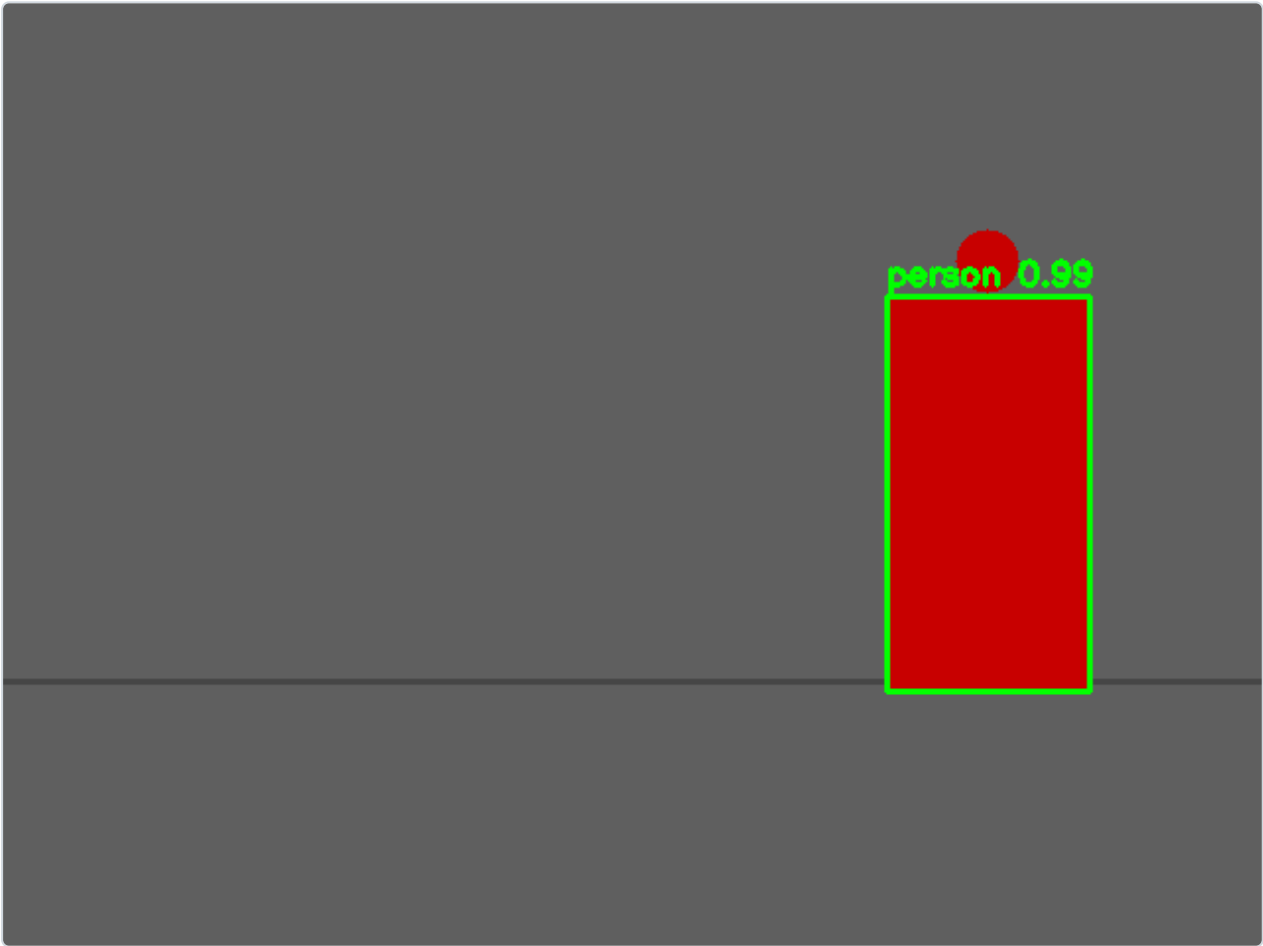


用合成画面把这三步拍成一张真实教学图（左：原图；中：红色掩码；右：绿框结果）：



HSV 三步分解：原图 → 红色掩码 → 绿框标注

对应真实运行截图（`source:=synthetic`，`backend:=hsv`）：



合成源 + HSV：红色剪影被绿框标成 person

你会在话题里看到类似这样的一条（真实捕获）：

```
detections:
- label: person
  confidence: 0.99
  cx: 0.216
  cy: 0.520
  width: 0.161
  height: 0.419
backend: hsv
inference_ms: 3.34
```

注意两件事：

- 1. **label 写成了 person**：HSV 并不认识「人」，它只认识「够大的红斑」。这里是故意把「红块」映射成下游契约里的 `person`，好让第五节跟随逻辑不用改字段。
- 2. **inference_ms ≈ 3 ms**：颜色阈值极快；后面 YOLO 动辄几百毫秒，对比会很扎眼。

2.2.3 HSV 的能力边界（什么时候必须上 YOLO）

场景	HSV 够不够	原因
课堂合成红块、色标物块分拣	够	颜色稳定、背景干净
穿红衣服的人在杂乱房间里	不够稳	灯光、阴影、其他红物干扰
同时认人、车、椅子、瓶子	不够	颜色无法表达「类别语义」
目标颜色会变（白天/夜晚、不同衣服）	不够	阈值要反复调

一句话：**HSV 看「颜色」，YOLO 看「长什么样」**。
下一小节我们把「安检员扫一眼大厅」的 YOLO 请上台。

2.3 主线②续：YOLO——看一眼报出全部

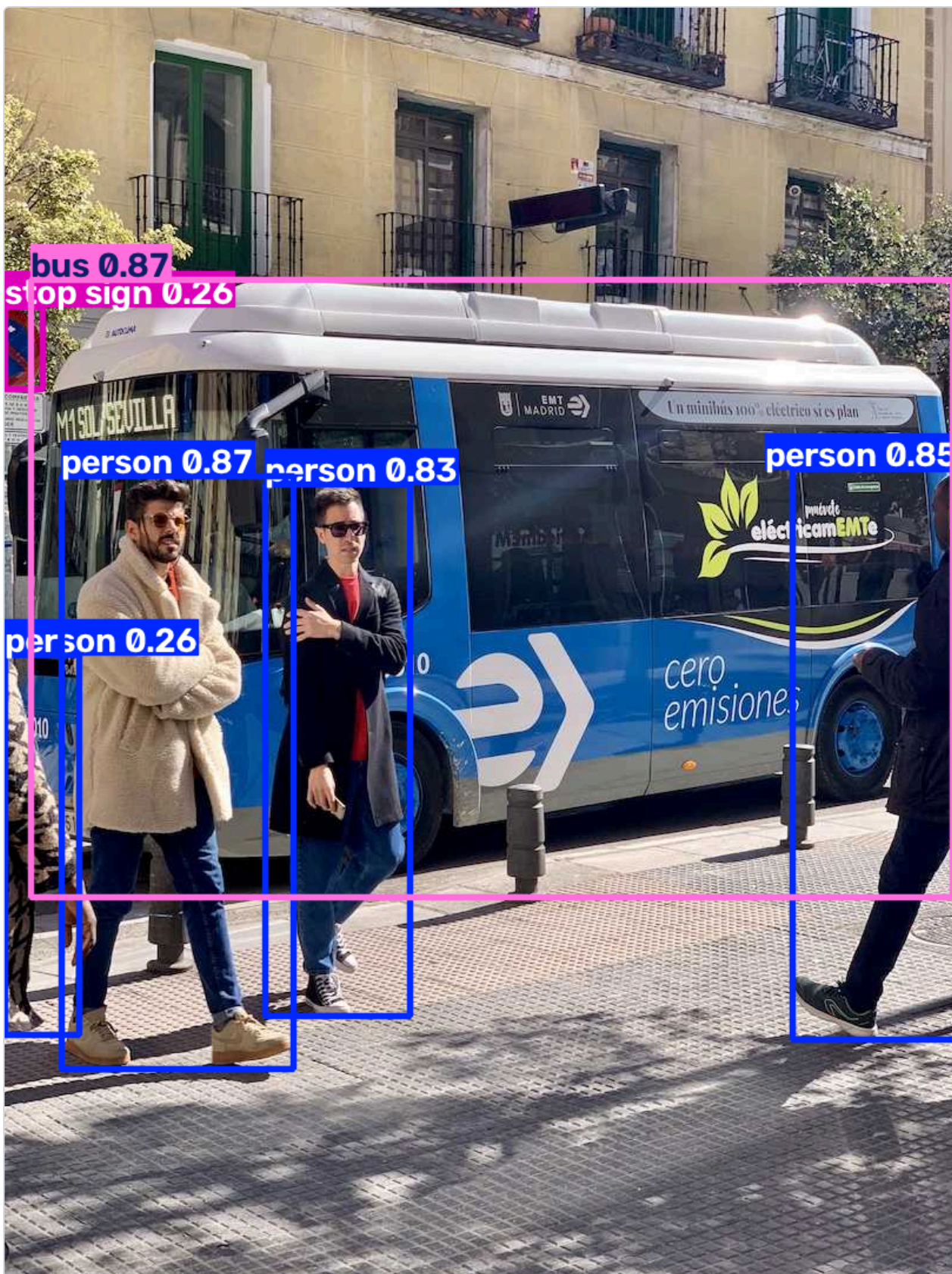
2.3.1 一个比喻：放大镜逐格扫 vs 安检员扫大厅

传统滑窗检测像安检员拿着放大镜，把画面切成成千上万个格子，逐格问「这里有人吗？」——慢，而且窗口大小还要试很多遍。

YOLO (You Only Look Once) 的名字本身就是宣言：整张图只过一遍网络，直接吐出所有目标的类别、置信度和框。
像安检员站在大厅门口，**一眼扫过去就报出**「左边三人、中间一辆巴士、置信度分别是……」。

本课使用 **YOLOv8n** (n = nano)：参数量约 300 万，适合笔记本 CPU 演示。更大的 `yolo8s/m/l` 更准但更慢，作业里可以自行替换。

用官方示例图 `assets/bus.jpg` 跑一次的真实结果：

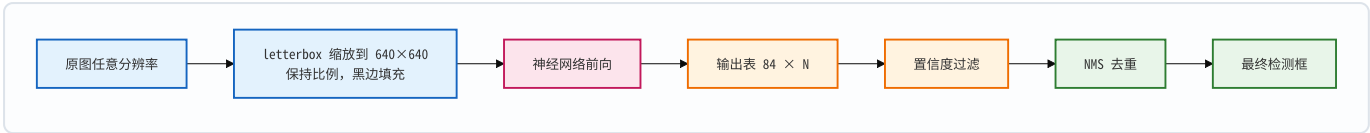


YOLOv8n 在 bus.jpg 上的检测结果：巴士 + 多人

可以看到：一辆 bus、多个 person，每个框旁有类别和置信度。这就是「看一眼报出全部」。

2.3.2 推理时网络里发生了什么（直觉版）

不必会背公式，先建立流水线直觉：



逐项用人话解释：

1. letterbox

网络训练时输入固定为 640×640 。若你直接把 1280×720 硬拉成正方形，人会被「压扁」，识别变差。letterbox 的做法是：等比缩放，多出来的边用灰色/黑色填充——像把照片贴进固定相框，上下或左右留白。

2. 输出表 $84 \times N$

YOLOv8 检测头会给出一大张候选表。对每个候选：

- 前 4 个数：框的位置与大小（在 640 画布上）；
- 后面 80 个数：COCO 80 类各自的分数（person、bicycle、car、bus……）。

所以是 $4 + 80 = 84$ 行； N 是候选框数量（常见约 8400）。

导出 ONNX 后冒烟测试打印的形状 `(1, 84, 8400)` 就是这个约定。

3. 置信度阈值 = 门卫放行标准

分数低于阈值的候选直接丢掉。

- 调太低：路人、椅背、影子都进来（误检）；
- 调太高：侧脸、半遮挡的熟人也被拦（漏检）。

本课默认约 0.4~0.5，可按场景调。

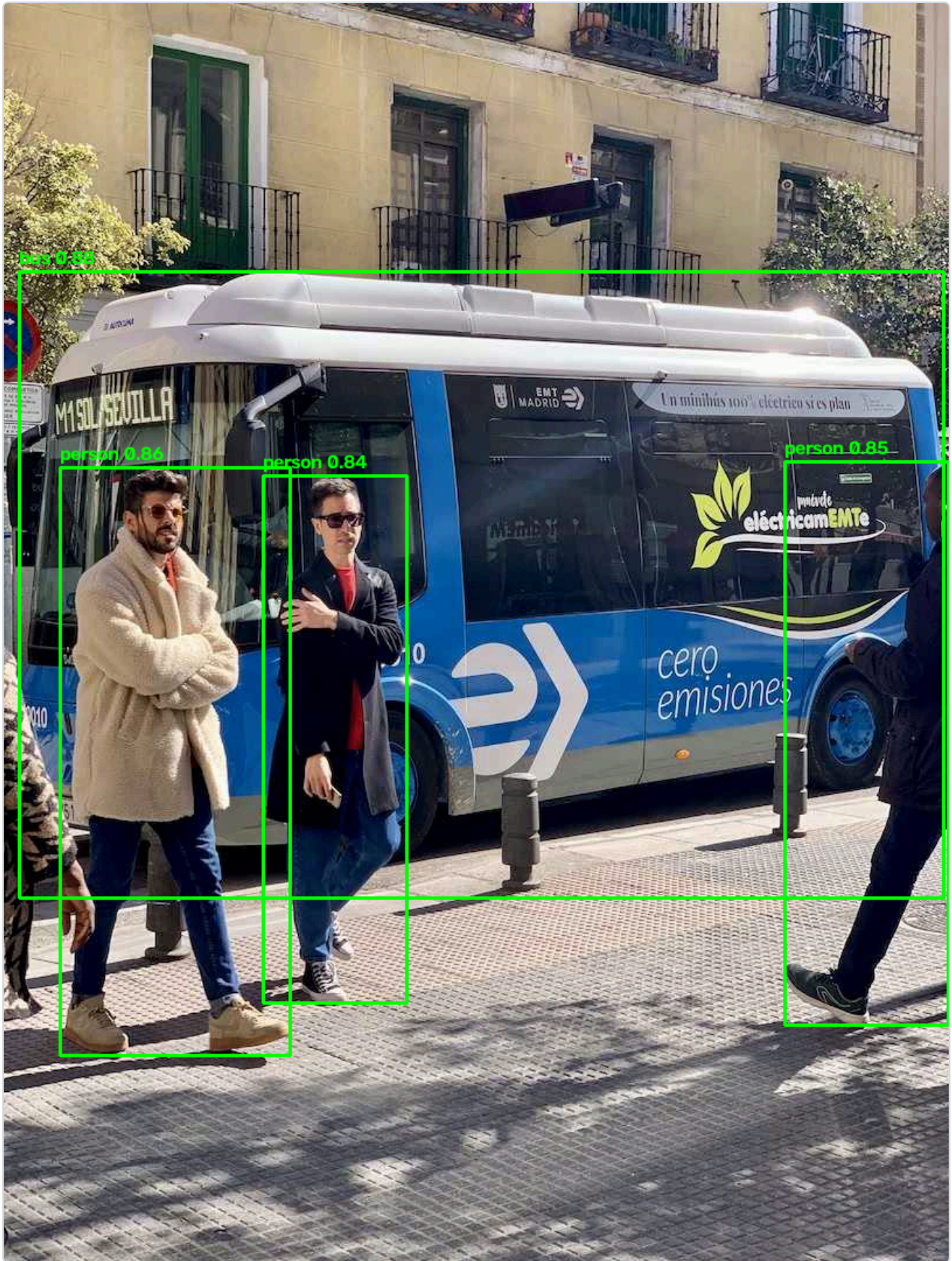
4. NMS（非极大值抑制）= 去重

同一个人可能被好几个高度重叠的框同时报中。NMS 留下分数最高的，把和它重叠太多的兄弟框删掉——会议室里很多人同时举手发言，只留声音最大的那个。

2.3.3 本课两条 YOLO 后端长什么样

后端名	读什么权重	运行时依赖	课堂角色
ultralytics	.pt (PyTorch 权重)	ultralytics + torch	「官方全家桶」，开发最省事
onnx	.onnx (ONNX 模型)	只需 onnxruntime	「精简部署」，更接近上机/嵌入式

用同一段演示视频 `assets/yolo_demo.mp4` 跑 ultralytics 时的真实画面：

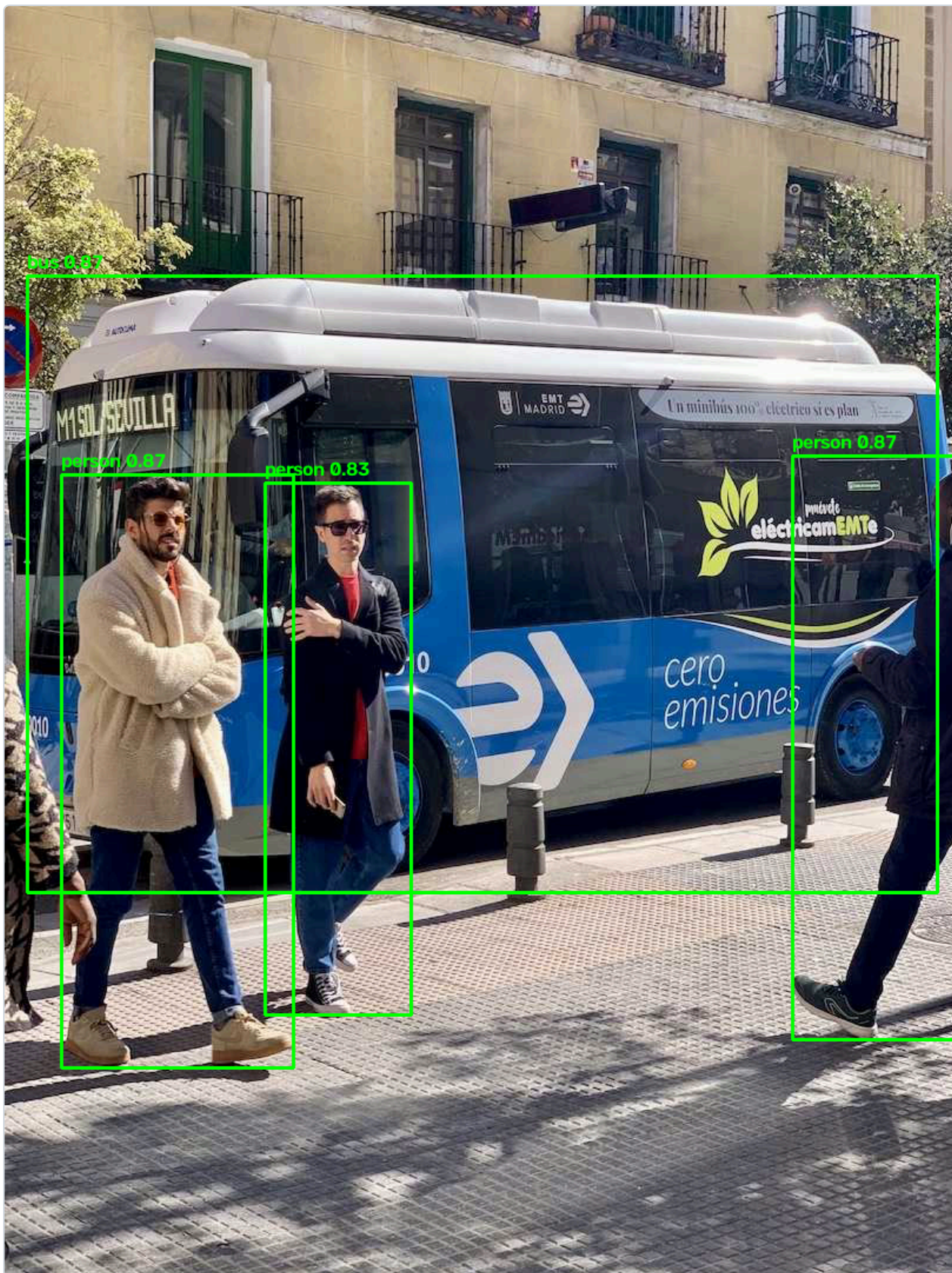


视频源 + ultralytics: 画面中人与巴士被标出

对应性能话题（真实捕获）：

```
{"backend": "ultralytics", "fps": 3, "inference_ms": 226.2, "dropped_frames": 175}
```

同一视频切到 onnx 后端时的真实画面：



视频源 + onnx: 同样能检出人与巴士

```
{"backend": "onnx", "fps": 2, "inference_ms": 616.13, "dropped_frames": 939}
```

```
detections:
- label: person
  confidence: 0.874
  ...
- label: bus
  confidence: 0.868
  ...
backend: onnx
inference_ms: 584.76
```

在本机 CPU 上 onnxruntime 未必比 ultralytics 更快（上表里甚至更慢）——ONNX 的价值首先是「依赖瘦」和「跨语言部署」，不是保证每一台 PC 都加速。有 NVIDIA GPU / TensorRT 时，ONNX 路线的优势才会更明显。

数字会随机器波动；讲义里的截图与 echo 来自同一套采集环境，用来建立量级直觉：HSV 约几毫秒，YOLO 在 CPU 上往往是百毫秒级，于是 fps 掉到个位数、开始丢帧——第五节会专门处理「推理跟不上采集」的问题。

2.4 主线③：模型怎么装进这台电脑？——.pt 与 .onnx

2.4.1 先搞清三个「文件后缀」

学员最容易被后缀吓住，这里一次性说清：

后缀 / 名词	它是什么	需要什么运行时	课堂类比
.pt	PyTorch 保存的权重（本课 yolov8n.pt）	PyTorch + ultralytics	私房菜谱：只有自家厨房（torch 生态）能完整复现
.onnx	开放神经网络交换格式（本课 yolov8n.onnx）	主要是 onnxruntime	标准菜谱：翻译成通用写法，别的厨房也能照做
HSV	不是模型文件，是一段颜色算法	只需 OpenCV	手电筒：没有菜谱也能摸黑照亮

ONNX 全称 Open Neural Network Exchange。你可以把它理解成「模型界的 PDF」：训练框架（PyTorch、TensorFlow.....）各写各的，导出成 ONNX 后，用统一的推理引擎跑。

2.4.2 为什么课堂要「翻译」成 ONNX



导出在本仓库里已经写成脚本：

```
python3 scripts/export_yolo_onnx.py
# 默认读 models/yolov8n.pt, 写出 models/yolov8n.onnx, 并用 onnxruntime 冒烟一次
```

真实导出日志里的关键几行（编码无关紧要，看数字与形状）：

```
PyTorch: starting from 'models/yolov8n.pt' with input shape (1, 3, 640, 640)
ONNX: export success, saved as 'models/yolov8n.onnx' (12.2 MB)
[check] onnxruntime 推理成功, 输出形状: (1, 84, 8400)
```

记住三个检查点：

1. 输入约定：`1×3×640×640`（batch、BGR 三通道、高、宽）；
2. 输出约定：`1×84×8400`（上面 § 2.3.2 讲过）；
3. 文件从约 6 MB 的 `.pt` 变成约 12 MB 的 `.onnx` —— 体积可能变大，但运行时依赖变瘦。

2.4.3 什么时候用哪条路

你的情况	建议
第一次课、只想看见框	先别导出，直接 <code>backend:=hsv</code> 或装好 <code>ultralytics</code> 后用 <code>.pt</code>
想少装包、或准备往板子上部署	导出 ONNX，用 <code>backend:=onnx</code>
公司电脑禁装 torch	ONNX + <code>onnxruntime</code> 往往更易过安检
什么 ML 包都装不了	完全没问题 ：HSV 兜底，CP4 照样能过

2.5 主线④：结果怎么交给 ROS 2？

检出框之后，如果只在本进程里 `cv2.imshow`，别的节点（跟随、语音播报、日志）都拿不到。

ROS 2 的做法是：**自定义消息 + 标准话题**，让「眼睛」和「腿」解耦。

2.5.1 一条检测：Detection2D

文件：`src/mecamind_interfaces/msg/Detection2D.msg`

字段	类型	含义	例子
<code>label</code>	<code>string</code>	类别名	<code>person / bus</code>
<code>confidence</code>	<code>float32</code>	置信度 0~1	<code>0.87</code>
<code>cx</code>	<code>float32</code>	框中心 x，归一化	<code>0.5</code> = 画面水平正中
<code>cy</code>	<code>float32</code>	框中心 y，归一化	<code>0.5</code> = 画面竖直正中
<code>width</code>	<code>float32</code>	框宽，归一化	<code>0.2</code> = 约占画面宽度 20%
<code>height</code>	<code>float32</code>	框高，归一化	<code>0.4</code> = 约占画面高度 40%

为什么坐标要归一化到 0~1?

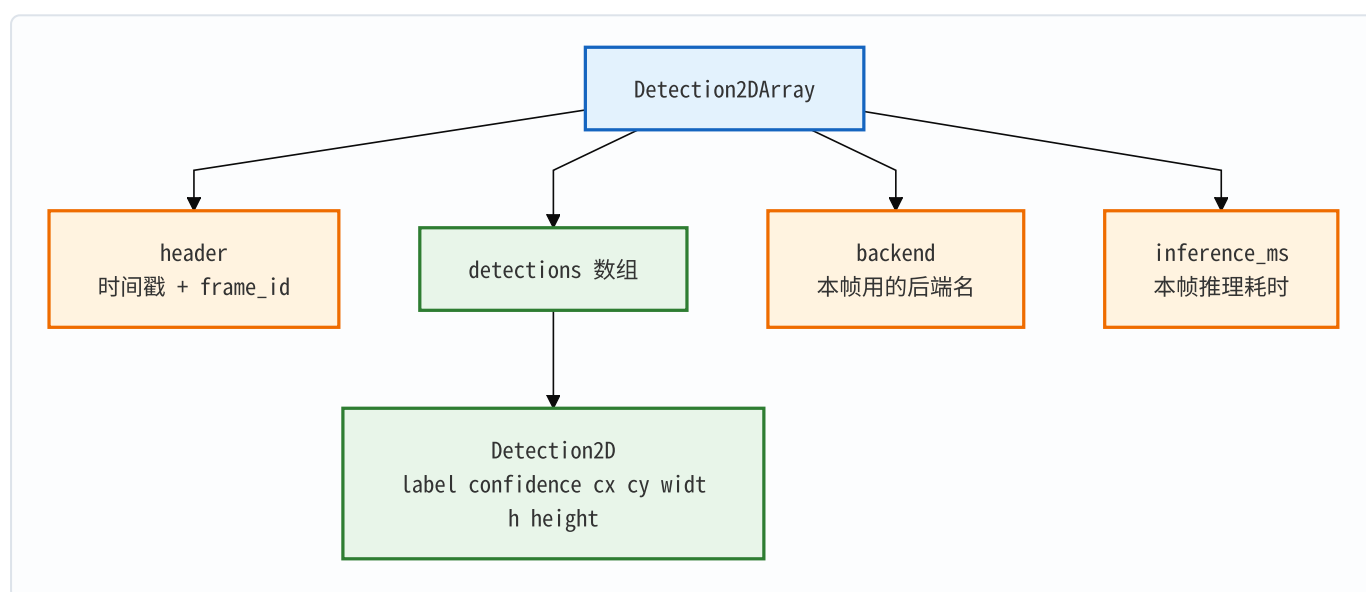
错误示范：下游写死「目标在第 512 像素 → 向右转」。换一台 1280 宽的相机，512 就变成「偏左」了，控制律全乱。

正确示范：报「目标在画面水平 70% 处」。无论 640 还是 1920，偏差都是 `cx - 0.5`，第五节的跟随 PID 不用改一行。

类比：问路时说「目标在你视野偏右两成」，而不是「在第 512 号地砖」——换一条街仍然成立。

2.5.2 一帧结果：Detection2DArray

文件：`src/mecamind_interfaces/msg/Detection2DArray.msg`



真实 HSV 一帧（已捕获）：

```
header:
  stamp: {sec: 1784797601, nanosec: 443607331}
  frame_id: camera_optical_frame
detections:
- label: person
  confidence: 0.99
  cx: 0.216
  cy: 0.520
  width: 0.161
  height: 0.419
backend: hsv
inference_ms: 3.34
```

`header.stamp` 用的是**图像采集时刻**（不是发布时刻），下游可以算端到端延迟。

`backend` / `inference_ms` 方便你一眼看出「这帧到底是手电筒还是神经网络在干活」。

2.5.3 检测节点还发布什么

话题	类型	用途
/mecamind/detections	Detection2DArray	主契约，结构化结果
/mecamind/detection_image	sensor_msgs/Image	画了绿框和文字的调试图
/mecamind/perception_metrics	std_msgs/String JSON	fps、inference_ms、dropped_frames

调试图用绿框，方便人眼验收；metrics 里的 `dropped_frames` 不是报错，而是「推理跟不上采集」的健康信号——第五节会展开。

2.5.4 JSON 桥：为什么还要一层「翻译」

第四节引入了漂亮的结构化消息，但第三节之前的下游（`perception_filter`、以后的跟随控制器）吃的是 **JSON 字符串** 话题 `/mecamind/raw_detections`。

工程上有两种升级法：

- a) 把所有下游一次性改成新消息——改动面大，一节课塞不下；
- b) 加一个适配器，新旧并行、逐步迁移。

本课选 **b**，节点名 `mecamind_detection_json_bridge`：

```
Detection2DArray → JSON String
/mecamind/detections → /mecamind/raw_detections
```

真实桥接输出长这样：

```
{"detections": [{"label": "person", "confidence": 0.99, "bbox": {"cx": 0.7039, "cy": 0.5198, "width": 0.1609, "height": 0.4187}}]}
```

这就是软件工程里的 **Adapter 模式**：一次课只动一层，出问题也好定位。

2.5.5 感知过滤：连续帧确认，别追幽灵

`perception_filter` 订阅 `/mecamind/raw_detections`，发布 `/mecamind/perception_event`。

它做两件事：

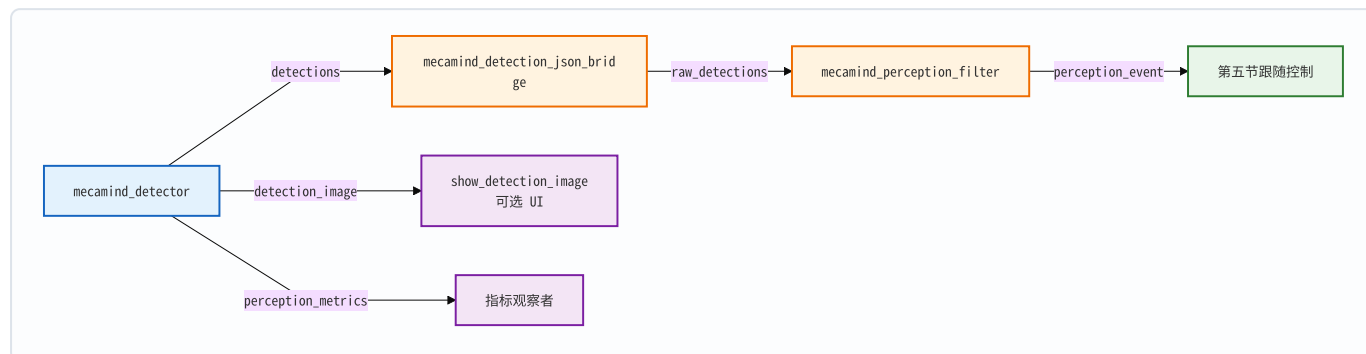
1. 按 `target_label` + `min_confidence` 挑出本帧最佳候选；
2. 连续 `N` 帧都看到，才把 `confirmed` 设为 `true`。

真实事件（HSV 链路捕获）：

```
{"label": "person", "confidence": 0.99, "cx": 0.5055, "cy": 0.5198, "width": 0.1609, "height": 0.4187, "confirmed": true, "confirm_count": 179}
```


`confirm_count: 179` 表示已经连续确认了很多帧——合成红块很稳，过滤器几乎一直放行。
若某帧突然闪出一个假人，连续确认会把它挡在跟随控制器之外，避免机器人去追幽灵。

整条节点关系：



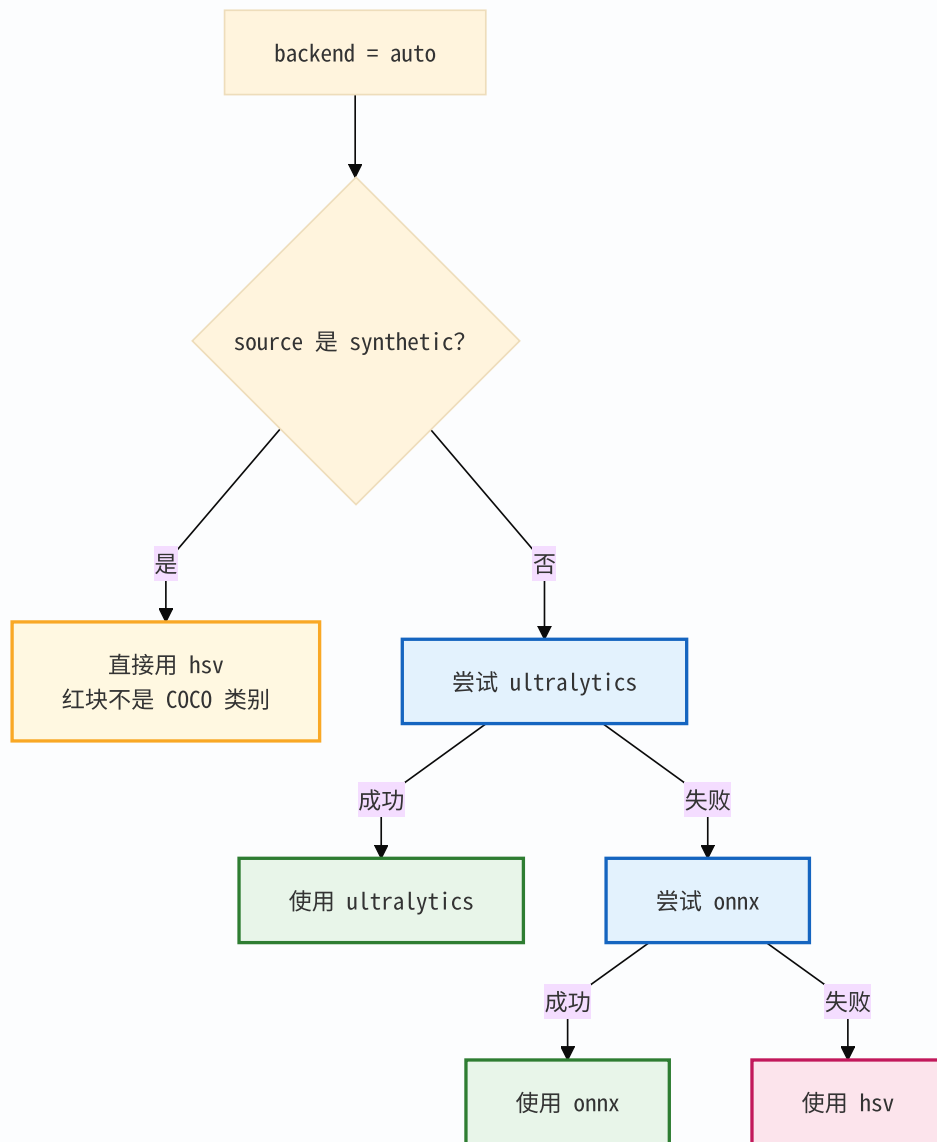
默认 `perception.launch.py` 会把上面三个核心节点一条命令拉起；`with_ui:=true` 时再加 OpenCV 看图窗口。

2.6 支撑细节：auto 降级，以及一个真实踩过的坑

2.6.1 后端选择决策树

参数 `backend` 可以是 `auto` / `ultralytics` / `onnx` / `hsv`。

`auto` 时按「市电 → 柴油机 → 手电筒」尝试：



对应源码逻辑在 `detector_node.py` 的 `_pick_backend`：缺包或模型加载失败会 `warn` 并试下一个，保证节点总能起来。

2.6.2 反面教材：装了 ultralytics，合成源反而「看不见人」

开发本课时真实踩过：

1. 默认 `source:=synthetic`，画面是左右晃的**红方块**；
2. 学员装好 `ultralytics` 后，`backend:=auto` 优先选了 YOLO；
3. YOLO 在 COCO 上没见过「纯色红矩形」，输出空检测；
4. 学员困惑：「我明明装了最强后端，怎么框都没了？」

修复策略（已写进代码）：

`auto + synthetic` 时强制走 HSV，并打日志说明原因。

这不是 YOLO 坏了，是输入契约和模型训练分布不匹配。

课堂口诀：

- 练颜色算法 → synthetic + hsv;
- 练真神经网络 → video/camera + ultralytics/onnx;
- 不要用红方块考核 YOLO。

2.6.3 多线程与「只留最新帧」(点到为止)

检测节点把采集和推理拆成两个线程，中间用容量为 1 的 LatestFrameQueue：新帧来时若队列满，挤掉旧帧并 dropped += 1。

直觉：安检仪传送带太快、安检员手慢时，与其堆积过期行李，不如永远只看最新那一件。
完整的多线程设计、跟随控制与丢帧治理，留给 10.3.5；本节只要知道：metrics 里的 dropped_frames 很大，通常表示 YOLO 在 CPU 上偏慢，不是节点崩溃。

2.7 速查

新名词表

名词	一句话
HSV	色相/饱和度/明度，颜色阈值检测常用空间
YOLO	You Only Look Once，单次前向出全部框
.pt	PyTorch 权重文件
.onnx	跨框架模型交换格式
onnxruntime	跑 ONNX 的轻量推理引擎
letterbox	等比缩放 + 填边，喂进固定尺寸网络
NMS	去掉高度重叠的重复框
归一化坐标	用 0~1 比例表示位置，与分辨率无关
CP4	本节自动验收脚本检查点

本课关键文件

路径	职责
src/mecamind_interfaces/msg/Detection2D.msg	单框消息
src/mecamind_interfaces/msg/Detection2DArray.msg	一帧结果 + 元信息
src/mecamind_perception/.../detector_node.py	检测节点
src/mecamind_perception/.../json_bridge_node.py	新消息 → 旧 JSON
src/mecamind_tools/.../perception_filter.py	连续帧确认
src/mecamind_bringup/launch/perception.launch.py	一条命令拉起链路
scripts/export_yolo_onnx.py	.pt → .onnx
scripts/test_cp4.sh	CP4 验收
scripts/show_detection_image.py	订阅调试图弹窗
models/yolov8n.pt / models/yolov8n.onnx	权重与导出模型
assets/bus.jpg / assets/yolo_demo.mp4	课堂素材

默认起来后能看到的节点与话题

真实 `ros2 node list` (HSV 默认链路):

```
/mecamind_detector
/mecamind_detection_json_bridge
/mecamind_perception_filter
```

真实 `ros2 topic list` 中与本课相关的:

```
/mecamind/detection_image
/mecamind/detections
/mecamind/perception_event
/mecamind/perception_metrics
/mecamind/raw_detections
```

HSV 模式下 `/mecamind/detections` 的 hz 约 **15 Hz** (与 `capture_rate_hz` 同量级); YOLO CPU 模式常降到数 Hz。

3. 启动前准备

3.1 构建（每次改消息或 Python 包后）

```
cd ~/mecamind_ros2
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install --packages-select \
  mecamind_interfaces mecamind_perception mecamind_tools mecamind_bringup
source install/setup.bash
```

`--symlink-install` 让你改 `.py` 后不必反复全量安装；但有一个坑见 § 3.3。

3.2 可选：安装 ML 依赖（想跑真 YOLO 时）

不装也能上课——默认会落到 HSV。若要跑 `ultralytics / onnx`：

```
./setup/install_dependencies.sh --with-ml
```

等价于：

```
pip install --user --break-system-packages "setuptools<80" ultralytics onnx onnxruntime
```

Ubuntu 24.04 的系统 Python 受 **PEP 668** 保护，直接 `pip install xxx` 常会报 `externally-managed-environment`。课程脚本用 `--user --break-system-packages` 装到用户目录，与现有 `ultralytics` 安装方式一致。

确认：

```
python3 -c "import ultralytics, onnx, onnxruntime; print('ML ok!')"
ls -lh models/yolov8n.pt models/yolov8n.onnx 2>/dev/null || echo "权重稍后在 LAB 里处理"
```

若还没有 `yolov8n.pt`，第一次用 `ultralytics` 时它可能自动下载；也可把课程提供的权重放到 `models/yolov8n.pt`。

3.3 两个必知坑位

症状	原因	处理
<code>colcon --symlink-install</code> 报 <code>setup.py develop</code> 相关错误	<code>setuptools>=80</code> 移除了该接口	<code>pip install --user --break-system-packages "setuptools<80"</code>
<code>pip install</code> 报 <code>externally-managed-environment</code>	PEP 668	用课程脚本的 <code>--user --break-system-packages</code> ，或自建 <code>venv</code> （进阶）

4. LAB 1: 零依赖跑通——synthetic + HSV

目标：不装任何 ML 包，看见绿框、读到结构化检测、看到 confirmed 事件。

4.1 启动

终端 A：

```
cd ~/mecamind_ros2
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch mecamind_bringup perception.launch.py \
  source:=synthetic backend:=hsv with_ui:=true
```

无显示器 / CI 时把 `with_ui:=false`。

日志里应出现类似：

```
MecaMind detector ready: source=synthetic, backend=hsv
MecaMind detection JSON bridge ready
```

4.2 看节点与话题

终端 B：

```
source /opt/ros/jazzy/setup.bash
source ~/mecamind_ros2/install/setup.bash
ros2 node list
ros2 topic list | grep mecamind
```

期望节点三个：`mecamind_detector`、`mecamind_detection_json_bridge`、`mecamind_perception_filter`。

4.3 读三条「眼睛」输出

```
ros2 topic echo /mecamind/detections --once
ros2 topic echo /mecamind/raw_detections --once
ros2 topic echo /mecamind/perception_event --once
ros2 topic echo /mecamind/perception_metrics --once
```

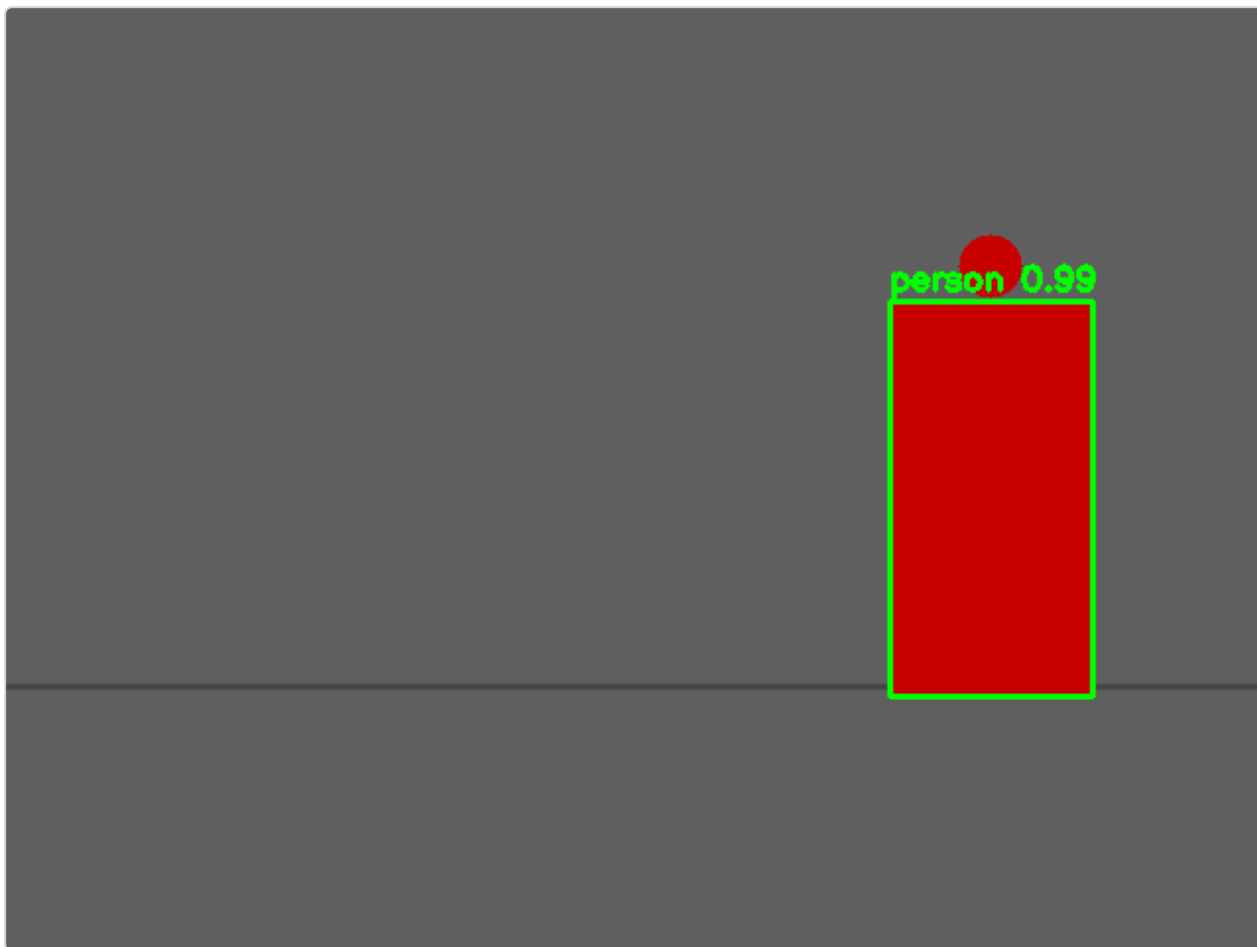
对照讲义 § 2.5 的样例：`backend: hsv`，`inference_ms` 个位数，事件里 `confirmed: true`。

调试图也可以不弹窗、只订阅保存（或用课程脚本）：

```
python3 scripts/show_detection_image.py
# 或
ros2 topic hz /mecamind/detections
```

本机实测 hz 约 14.9 (接近 15)。

4.4 你应当看到的画面



LAB1 期望画面：灰底红剪影 + 绿色 person 框

过关标准：窗口或调试图里有绿框；`detections` 非空；`perception_event` 出现 `confirmed: true`。

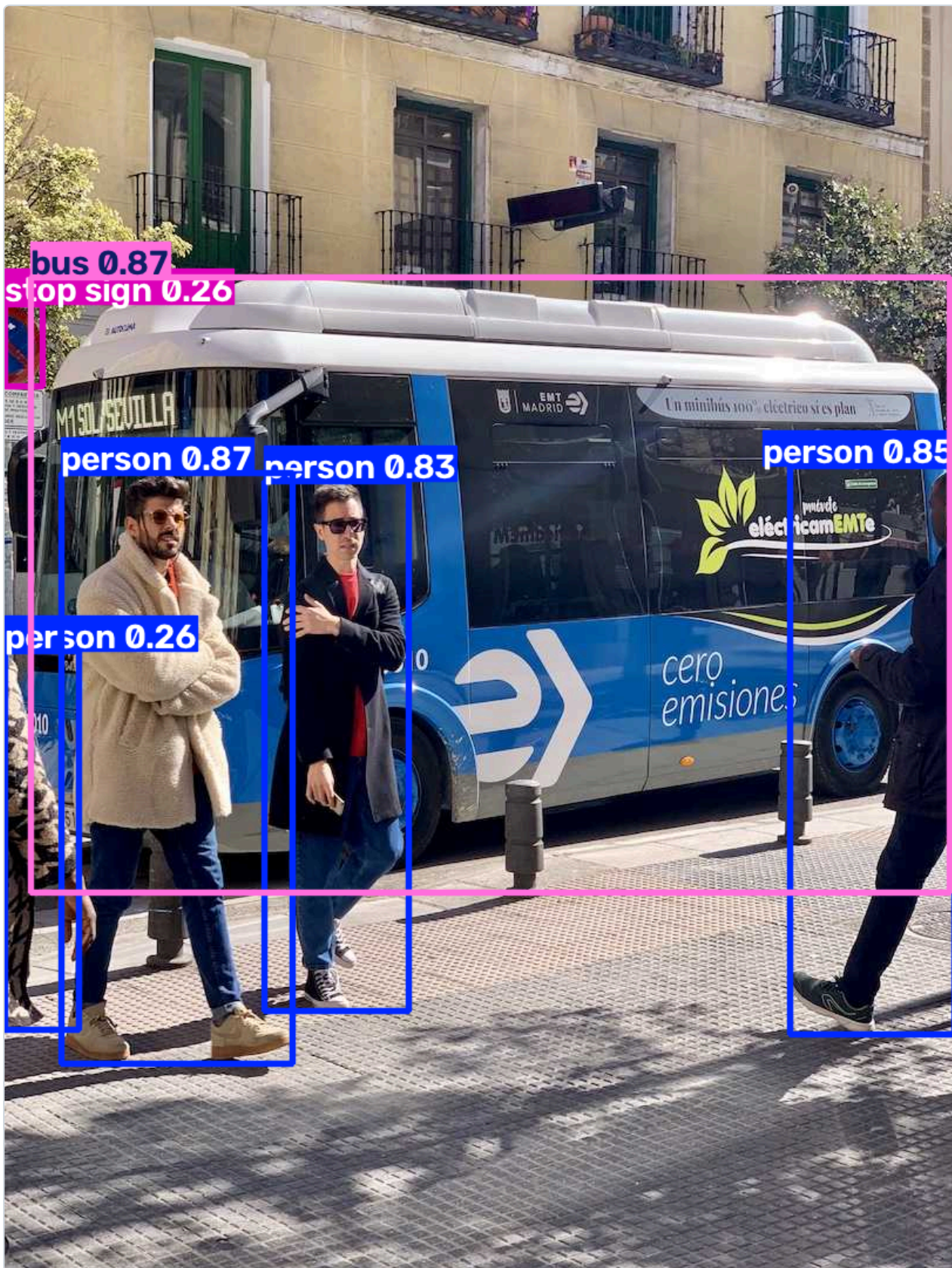
5. LAB 2：真实 YOLO——单图直觉 + 视频源 ultralytics

目标：亲手确认「神经网络认的是语义目标，不是红块」；再用演示视频跑 ROS 链路。

5.1 单图体验（不必起 ROS）

```
cd ~/mecamind_ros2
python3 - <<'PY'
from ultralytics import YOLO
model = YOLO("models/yolov8n.pt")
r = model("assets/bus.jpg")[0]
r.save("/tmp/bus_out.jpg")
for box in r.boxes:
    print(f"{r.names[int(box.cls[0])]:12s}  conf={float(box.conf[0]):.2f}")
print("saved /tmp/bus_out.jpg")
PY
```

对照讲义中的标注图：



bus.jpg 上的 YOLO 标注

若这里就 ImportError, 回到 §3.2 装 ML 依赖。

5.2 视频接入 ROS

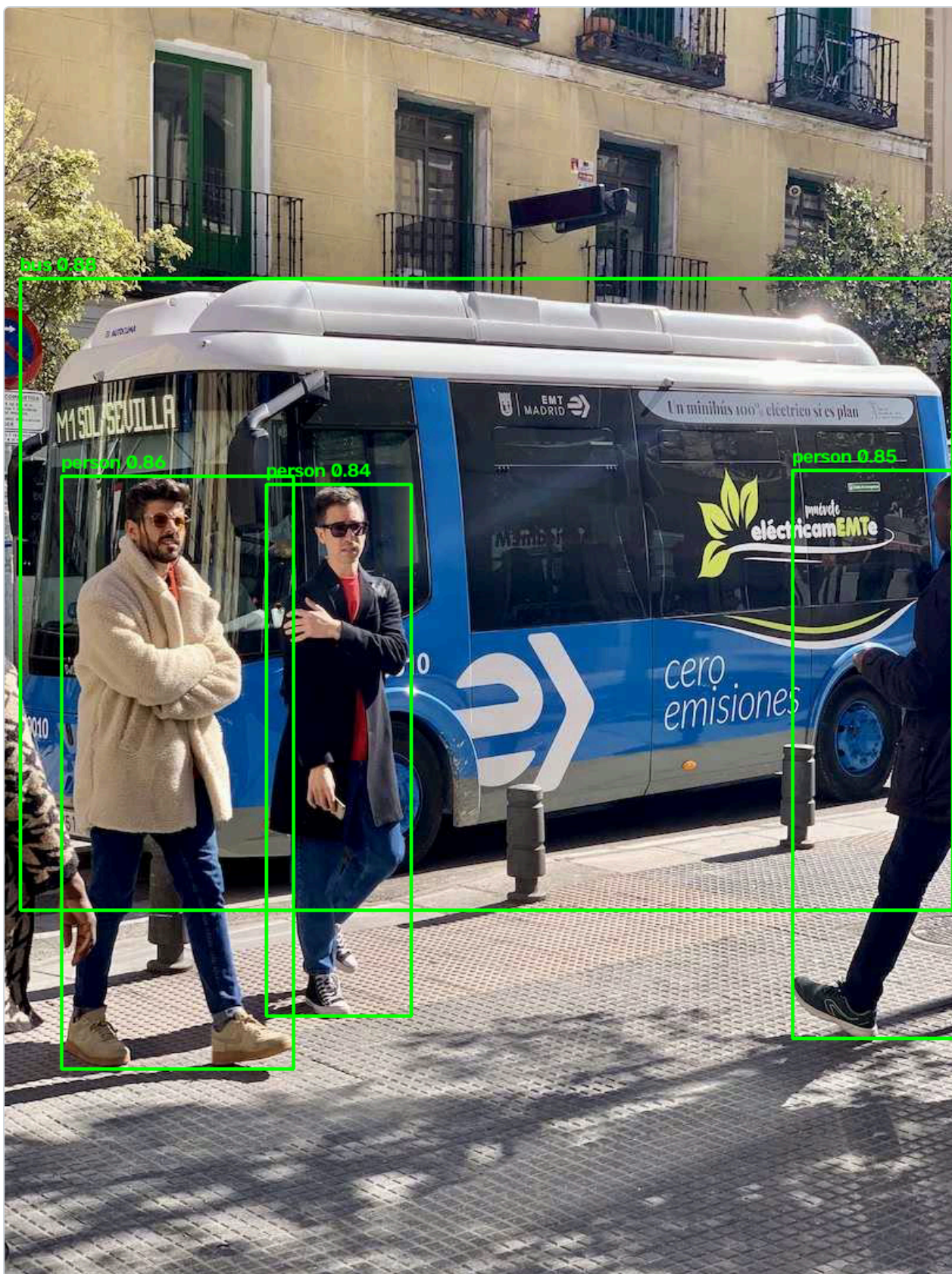
终端 A (先停掉 LAB1 的 launch, Ctrl+C):


```
ros2 launch mecamind_bringup perception.launch.py \  
  source:=video \  
  video_path:=$HOME/mecamind_ros2/assets/yolo_demo.mp4 \  
  backend:=ultralytics \  
  model_path:=$HOME/mecamind_ros2/models/yolov8n.pt \  
  with_ui:=true
```

终端 B:

```
ros2 topic echo /mecamind/detections --once  
ros2 topic echo /mecamind/perception_metrics --once
```

期望: backend: ultralytics, detections 里可能同时有 person、bus 等; inference_ms 往往是百毫秒级, fps 个位数。



LAB2 期望画面：视频帧上的 YOLO 绿框

本机一次 metrics 样例：

```
{"backend": "ultralytics", "fps": 3, "inference_ms": 226.2, "dropped_frames": 175}
```

`dropped_frames` 变大是正常现象：采集仍按约 15 Hz 塞帧，推理却要 200 ms+，队列只留最新帧，旧帧被挤掉。

过关标准：画面里出现对「人/车」等真实物体的框；metrics 的 backend 字段为 `ultralitics`。

6. LAB 3: ONNX 导出与推理

目标：完成一次「私房菜谱 → 标准菜谱」翻译，并用 onnx 后端跑通同一视频。

6.1 导出

```
cd ~/mecamind_ros2
python3 scripts/export_yolo_onnx.py
```

成功时末尾应类似：

```
[check] onnxruntime 推理成功，输出形状：(1, 84, 8400)
[export] 完成。检测节点用法：
ros2 launch mecamind_bringup perception.launch.py backend:=onnx model_path:=.../yolov8n.onnx
```

若中间出现 `onnxslim` / PEP 668 警告，只要最终 `export success` 且 `check` 通过，课堂可忽略（简化器是可选优化）。

确认文件：

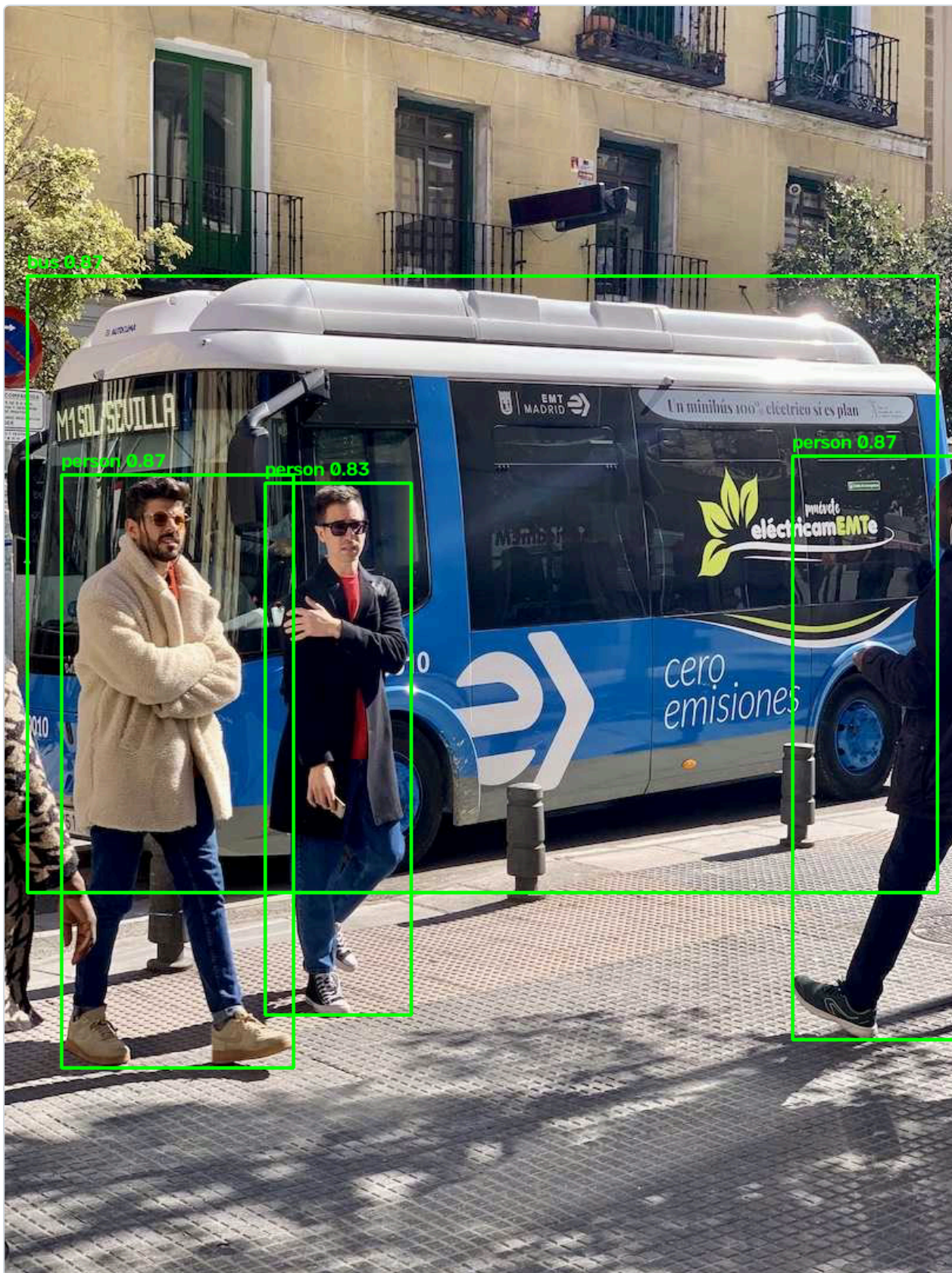
```
ls -lh models/yolov8n.onnx
```

6.2 用 onnx 后端跑视频

```
ros2 launch mecamind_bringup perception.launch.py \
  source:=video \
  video_path:=$HOME/mecamind_ros2/assets/yolo_demo.mp4 \
  backend:=onnx \
  model_path:=$HOME/mecamind_ros2/models/yolov8n.onnx \
  with_ui:=true
```

```
ros2 topic echo /mecamind/detections --once
ros2 topic echo /mecamind/perception_metrics --once
```

期望：`backend: onnx`，仍能检出 `person/bus`。



LAB3 期望画面：onnx 后端视频检测

本机一次 detections 片段：

```
- label: person
  confidence: 0.874
- label: bus
  confidence: 0.868
backend: onnx
inference_ms: 584.76
```

过关标准: `models/yolov8n.onnx` 存在; 话题里 `backend: onnx`; 画面有框。

7. LAB 4: Gazebo 相机接入 (可选加强)

目标: 机器人在仿真里真正「睁眼」。依赖第三节环境能起 Gazebo。

7.1 起仿真 (示例)

另开终端, 按你课堂惯用的 `bringup` 启动 Gazebo (需已有 `/camera/image_raw` 或等价话题):

```
# 示例: 以课程仿真入口为准, 确保相机话题在发布
ros2 topic list | grep -i camera
ros2 topic hz /camera/image_raw
```

7.2 感知订阅相机

```
ros2 launch mecamind_bringup perception.launch.py \
  source:=camera \
  camera_topic:=/camera/image_raw \
  backend:=auto \
  with_ui:=true
```

若已导出 ONNX, 可显式:

```
ros2 launch mecamind_bringup perception.launch.py \
  source:=camera \
  backend:=onnx \
  model_path:=$HOME/mecamind_ros2/models/yolov8n.onnx \
  with_ui:=true
```

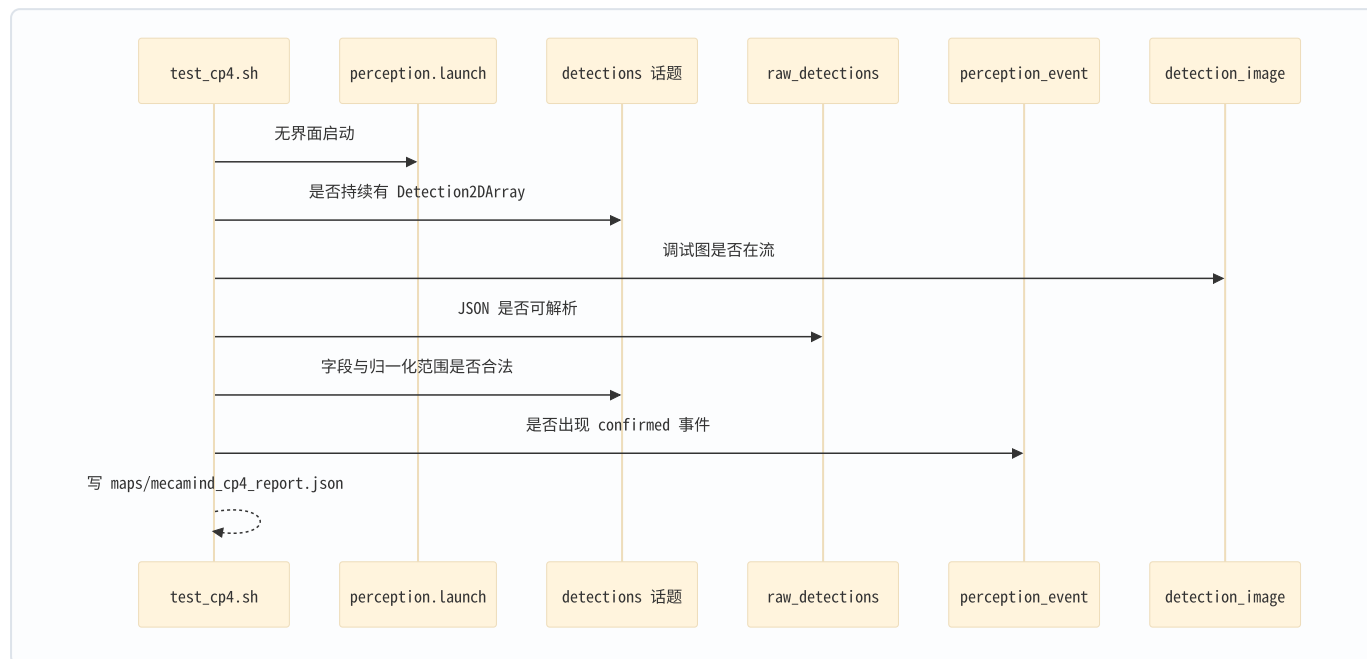
在 Gazebo 里把机器人对准「有人/有车」的场景 (或放一张贴图), 观察调试图。

过关标准: `source=camera` 时 `detections` 随相机画面变化; 扭动仿真相机, 框跟着目标走。

若本机暂时起不了 Gazebo, LAB 1~3 + CP4 已覆盖本节必会内容, 本 LAB 可课后补做。

8. CP4 自动验收

8.1 验收在查什么



六项检查（脚本输出里的「正常」项）：

1. detections_flowring
2. debug_image_flowring
3. json_bridge_flowring
4. detection_fields_valid
5. json_contract_compatible
6. filter_confirmed_event

全部通过后打印 MECAMIND_CP4_OK。

8.2 三条推荐命令

默认零依赖（synthetic → 实际 hsv）：

```
./scripts/test_cp4.sh
```

真实输出例：

```
[CP4] source=synthetic backend=auto model=yolov8n.pt video=无
[CP4] [正常] detections_flowring
...
[CP4] 推理后端: hsv, 20 秒收到 20 帧检测
MECAMIND_CP4_OK
```

ONNX + 视频:

```
MECAMIND_CP4_SOURCE=video \  
MECAMIND_CP4_VIDEO=assets/yolo_demo.mp4 \  
MECAMIND_CP4_BACKEND=onnx \  
MECAMIND_CP4_MODEL=models/yolov8n.onnx \  
./scripts/test_cp4.sh
```

真实输出例:

```
[CP4] 推理后端: onnx, 20 秒收到 20 帧检测  
MECAMIND_CP4_OK
```

Ultralytics + 视频:

```
MECAMIND_CP4_SOURCE=video \  
MECAMIND_CP4_VIDEO=assets/yolo_demo.mp4 \  
MECAMIND_CP4_BACKEND=ultralytics \  
MECAMIND_CP4_MODEL=models/yolov8n.pt \  
./scripts/test_cp4.sh
```

不要用 `SOURCE=synthetic` 去验 `ultralytics/onnx`——红方块不是 COCO 目标，检测可能为空，过滤器出不了 `confirmed`，CP4 会失败。这正是 § 2.6.2 的坑。

9. 常见问题

Q1: `pip install` 报 `externally-managed-environment`

系统 Python 启用了 PEP 668。用:

```
./setup/install_dependencies.sh --with-ml
```

或手动加 `--user --break-system-packages`。

Q2: `colcon build --symlink-install` 突然失败，提到 `setuptools / develop`

把 `setuptools` 钉回 79.x:

```
pip install --user --break-system-packages "setuptools<80"
```

Q3: 装了 `ultralytics`，默认 `launch` 却「检不到人」

你很可能在 `synthetic` 源上跑了 YOLO。看日志:

- 若 `backend=auto` 且 `source=synthetic`，应强制 `hsv`；
- 若你强行 `backend:=ultralitics source:=synthetic`，空检测是预期现象。

改用视频或相机源。

Q4: topic does not appear to be published yet

launch 后等 3~6 秒再 echo；确认 `source install/setup.bash` 过；用 `ros2 node list` 看三个节点是否都在。

YOLO 首次加载模型可能再多等十几秒。

Q5: onnx 比 ultralytics 还慢

CPU 上完全可能。ONNX 的课堂意义是依赖瘦、可移植；速度优势要结合 GPU / 厂商加速库才明显。先保证「能跑通」。

Q6: 导出时 onnxslim 安装失败

可忽略，只要出现 `export success` 且脚本内 `check` 通过。简化器不是硬依赖。

Q7: 调试图话题有、窗口没弹出

`with_ui:=false` 或没有 `DISPLAY`。无界面环境用 `ros2 topic echo` / 自行订阅存图即可；WSL 需确认 WSLg 或 X server。

Q8: CP4 提示已有验收在跑

`/tmp/mecamind_cp4.lock.d` 残留或并行跑了两份。等前一份结束，或确认无进程后删掉锁目录再跑（勿在别人正在验收时删）。

10. 本节验收清单

课上或课后自评：

- ☐ 能用自己的话复述四问主线（图像来源 / 找目标 / 部署 / ROS 输出）
 - ☐ 能解释 `.pt`、`.onnx`、HSV 的差别与三级兜底
 - ☐ LAB1: `synthetic+hsv` 看到绿框与 `confirmed` 事件
 - ☐ LAB2: `ultralitics` + 视频看到真实类别检测（若已装 ML）
 - ☐ LAB3: 导出 `onnx` 并用 `backend:=onnx` 跑通（若已装 ML）
 - ☐ `./scripts/test_cp4.sh` 打印 `MECAMIND_CP4_OK`
 - ☐ 知道为何坐标要归一化，以及 JSON 桥存在的原因
-

11. 课后作业

1. **换模型**：把 `model_path` 换成 `yolov8s.pt`（需自行下载），对比同一视频下 `inference_ms` 与检出数量，写三句话结论。
2. **调门卫**：把 `conf_threshold` 分别设为 `0.25` 与 `0.70`，截两张调试图，说明误检/漏检如何变化。
3. **过滤实验**：只改 `perception_filter` 的 `confirm_frames` (`1` vs `5`)，观察 `perception_event` 的响应速度与稳定性。
4. **(选做) 类别过滤**：在 `detector` 或 `filter` 侧只保留 `person`，让 `/mecamind/detections` 不再出现 `bus`——为第五节跟随做准备。

12. 下一课预告：10.3.5 多线程感知与视觉跟随

本节已经把「眼睛」接到了 ROS 2，并留下两个伏笔：

1. **推理慢于采集** → `dropped_frames` 上涨；需要认真对待最新帧队列与线程边界；
2. **confirmed 事件里有 `cx`** → 偏差 `cx - 0.5` 可以直接驱动角速度，让机器人跟着人走。

第五节会在本节消息契约之上，把检测、过滤、速度仲裁串成可演示的跟随链路，并讲清为何「容量为 1 的队列」反而是实时系统的朋友。

附录 A：本讲义截图与终端样例来源

以下均来自本仓库真实运行采集，便于对照复现：

文件	含义
<code>docs/images/10.3.4/detection_synthetic_hsv.png</code>	synthetic + hsv
<code>docs/images/10.3.4/detection_video_ultralytics.png</code>	video + ultralytics
<code>docs/images/10.3.4/detection_video_onnx.png</code>	video + onnx
<code>docs/images/10.3.4/hsv_pipeline_steps.png</code>	HSV 三步教学图
<code>docs/images/10.3.4/yolo_bus_annotated.png</code>	bus.jpg YOLO 标注

CP4 样例输出见正文 § 8.2；话题 `echo` 样例见 § 2.5 / LAB 各节。

文档版本：与仓库 `perception.launch.py` + `scripts/test_cp4.sh` 当前行为对齐。若参数名有变更，以 `launch` 文件与节点 `declare_parameter` 为准。